

Technical Milestone Report: Automatic Synthesis of Virtual Model Controllers for Robotic Manipulation

Hasan Shahrestani (smhs4)

January 22, 2026

Abstract

Summary. This project investigates whether *Virtual Model Controllers (VMCs)* for manipulation can be generated automatically rather than designed by hand. The current approach uses a genetic algorithm (GA) to search over both (i) the *structure* of a VMC represented as a network of virtual springs/dampers (tendons) between robot and object sites, and (ii) the continuous parameters (stiffness, damping, rest length). Progress to date includes: a working end-to-end GA training pipeline in MuJoCo; multiple iterations of the fitness function; improved genome encoding and genetic operators; parallel evaluation; and initial tests on pickup and mesh-based objects.

1 Motivation and project objectives

Robot manipulation remains difficult to engineer reliably because contact-rich interaction, friction variability, and object diversity make controller design highly task-specific [1]. Virtual Model Control offers an intuitive way to produce compliant motion by attaching *virtual* springs, dampers and masses to the robot and object, generating torques that mimic physical interaction [2]. However, hand-crafted VMCs are time-consuming to tune, custom-made per task, and do not scale well to more complicated behaviours.

1.1 Overall aim

The aim of this project is to automatically synthesise a VMC from a manipulation task

definition (e.g., *pick up the object*) by searching for:

- A virtual component **structure**: which parts of the robot and objects are connected to each other via springs and dampers.
- Continuous **parameters**: spring stiffness, damping and rest length.

1.2 Success criteria (milestone-level)

By the end of the project, success will be measured by:

- Demonstrating consistent ‘task completion’ in simulation across varied initial object poses (robustness).
- Producing controllers that transfer to at least one real-robot manipulation test with safety constraints. (sim to real)
- Providing a clear methodology: encoding, fitness design, and optimisation settings that can be reused and applied to other robot manipulation tasks.

2 Methodology

2.1 VMC Synthesis Pipeline

2.1.1 Simulation setup

Experiments are carried out in MuJoCo using an upper-body humanoid platform (Sciurus17) and a simplified grasping setup. The current testbed uses simplified end-effectors (“chopsticks” or planar grippers) and a small object to evaluate grasp and lift behaviours. The same pipeline is being extended to mesh-based objects.

Extension:

- Incorporate **soft or compliant materials** in simulation to improve gripping performance and better reflect real-world contact interactions.
- Enable **greater object diversity** during training by supporting deformable or semi-compliant objects, increasing robustness and generalisation.
- Explore **simulation speed optimisations**, by adjusting parameters to balance between realism and efficiency or exploring computational hacks such as using a GPU to increase speed.

2.1.2 VMC search space

A VMC candidate is represented as a set of tendons (virtual springs/dampers) between predefined *sites* (Fig. 3) on/relative-to the robot and the object. Each tendon is parameterised by:

- stiffness k and damping c (continuous),
- rest length ℓ_0 (continuous, optionally enabled),
- endpoints: indices of robot/object sites (discrete).
- number of spring and dampers in the controller

Extension: The search space is currently limited:

- Number of possible spring and dampers are fixed
- sites are predefined discrete points.
- Constant. Once set parameters do not change during operation. Parameters especially continuous ones should be time varying.

Finding efficient ways of searching the space while ensuring stability will allow these limitations to be reduced.

2.1.3 Optimisation algorithm

A genetic algorithm (GA) evolves a population of candidate controllers by repeated evaluation, selection, crossover and mutation [3]. Key GA hyperparameters include

- population size, number of generations, tournament size, crossover/mutation

probability and elitism.

- **encoding** which defines how each candidate VMC mechanism is represented, and dictates how they are crossed-over/mutated.
- **crossover/mutation mechanisms** which govern how encodings are manipulated and combined.

Extension: While the GA has demonstrated reliable convergence to effective VMC designs at reasonable computational cost, several extensions and alternative optimisation strategies may be explored.

- **Neural network-based controllers and Reinforcement learning approaches**, RL algorithms which are designed for fixed horizon, final reward scenarios such as GRPO [4] seems well suited for this task.
- **Hybrid methods**, combining GA-based structural optimisation with learning-based parameter refinement, could leverage the strengths of both global search and local optimisation.

2.1.4 Fitness function

Each task requires a fitness function to optimise so that quality of the proposed VMC can be quantified. In object manipulation [5] this is a multi-objective function and depends on the task itself. For example pickup task fitness currently combines task-specific signals such as **final object height** but general quality objective still remain: stability, efficiency, contact and robustness are all necessary for a high grasp.

Extensions: The current fitness function combines multiple criteria using a linearly weighted sum with manually tuned weights. While effective, this approach can be improved in several ways:

- **Objective normalisation:** scale individual fitness terms to comparable ranges to reduce sensitivity to arbitrary weighting and improve optimisation stability.
- **Multi-objective optimisation:** explicitly search for Pareto-optimal solu-

tions (e.g. multi-objective GA concepts (e.g., NSGA-II) [6]) rather than collapsing all objectives into a single scalar fitness.

- **Structural constraints:** restrict the search space a priori by enforcing stability- and efficiency-aware design rules, reducing reliance on fitness penalties for infeasible or pathological solutions.

2.2 Practical Implementation

- **Object tracking and perception:**

The robot must estimate the object’s position, orientation, and (approximate) geometry to correctly instantiate the VMC and compute interaction forces. The Sciurus17 platform provides a 3D point-cloud camera that may be used to reconstruct object geometry and localise contact points. Alternative sensing solutions include classical computer vision pipelines or external motion-capture systems (e.g. OptiTrack) for higher accuracy.

- **Mapping MuJoCo tendons to robot torques:** The current MuJoCo model relies on built-in tendon abstractions applied to a gravity-compensated robot. On real hardware, equivalent behaviour must be achieved by converting virtual tendon forces into joint torques via an inverse Jacobian. Implementing this mapping explicitly in simulation would reduce the sim-to-real gap and better reflect the physical control pipeline. This is also required to match the torque limits of the robot. Currently induced torque is not calculated and therefore simulation results may not match reality.
- **Encoding translation and deployment:** A software tool is required to automatically convert the optimised GA encoding into a Julia-compatible VMC representation that can be executed on the robot. At present, this translation is performed manually and represents

a clear opportunity for automation and improved reproducibility.

- **Real-world fine-tuning:** While large-scale GA optimisation is impractical on hardware due to sample inefficiency, limited real-world refinement may be performed after simulation training. This could include parameter fine-tuning or local adaptation to unmodelled dynamics, improving robustness without prohibitive data collection.

3 Progress to date

Significant progress has been made across four key milestones:

- **Pipeline Development:** Established an end-to-end GA pipeline in MuJoCo. This includes parallel fitness evaluation, automated site/tendon generation, and support for mesh-based geometries (Fig. 1).
- **Genome Redesign:** Transitioned to a fixed-size matrix encoding (Fig. 4). By limiting the number of virtual components, we reduced search space growth and improved the effectiveness of genetic operators.
- **Algorithmic Refinement:** Implemented Simulated Binary Crossover (SBX) for continuous parameters and discrete crossover for topology. Robustness was enhanced by evaluating candidates across randomized initial object poses.
- **Preliminary Results:** Achieved consistent lifting for primitive blocks and pyramids. Results are also quantitatively assessed by viewing fitness function, improvements and maintaining population diversity (Fig. 2). Current work is focused on refining contact stability for complex meshes.

3.1 Lessons learned

- **Search space / instability:** increasing tendon count and permissive encodings can produce unstable, unphysical

behaviours.

- **Fitness shaping:** small changes to weighting can dominate selection and push the GA toward degenerate solutions (e.g., “gaming” centre/height terms).
- **Generalisation:** optimisation on a single initial pose can lead to brittle controllers; evaluating across multiple initial conditions improves robustness but increases compute cost.
- **GA parameter tuning:** Population size and generation number is an important factor in performance but comes at a computational cost. Mutation and cross over probability also need to be tuned to the size of the genome.

4 Plan for remaining work

The remainder of the project is prioritised around achieving a *minimum viable sim-to-real transfer* for a single representative object and task, before expanding to robust and generalisable performance. VMC synthesis for simple task and general mesh objects are ready; therefore the critical path is implementing the practical deployment steps required for execution on the Sciurus17 robot, validating equivalence between simulation and hardware, and establishing safe repeatable trials.

4.1 WP1: Minimum viable sim-to-real transfer (highest priority)

Goal: execute a GA-trained VMC on the real robot for one target object and manipulation task, under safety constraints, with behaviour consistent with simulation. The steps for this are mentioned section 2.2 Practical Implementation.

Timeline - Weeks 1 & 2 Lent

- **Object tracking and state estimation.** Implement the minimum sensing pipeline required to estimate object

pose (and optionally velocity) robustly enough to instantiate VMC sites.

- **Explicit VMC-to-torque mapping.** Replace any MuJoCo-specific abstractions with an explicit mapping from virtual component forces to joint torques via robot Jacobians.
- **Genome-to-deployment automation.** Build a reproducible conversion from the GA genome to a deployable VMC configuration (Julia-compatible controller representation), eliminating manual transcription.
- **Safety envelopes and staged deployment.** Define conservative torque/velocity limits and termination conditions.

4.2 WP2: Fine-tuning and generalisation after first real success

Goal: improve reliability and reduce sensitivity to modelling errors, and object initial pose and size after the first working sim-to-real demonstration.

Timeline - Week 3 & 4 Lent

- **Generalisation to more tasks and shapes:** Test on more complicated shapes and more sophisticated tasks.
- **Hardware calibration loop.** Identify which parameters cause the largest sim-to-real discrepancy (friction, compliance, controller gains, latency) and calibrate them using short diagnostic experiments.
- **Lightweight real-world refinement.** Introduce limited on-hardware tuning that is sample-efficient (e.g. local parameter search around the GA solution, or constrained updates), while keeping the bulk optimisation in simulation.
- **Robustness training in simulation.** Increase robustness by evaluating candidate controllers across distributions of initial poses and physical parameters (mass, friction), and enforce constraints aligned with the hardware safety limits.

4.3 WP3: Improved optimisation and adaptive control (stretch goals)

Goal: move beyond “set-and-go” controllers and object-specific optimisation towards controllers that generalise and adapt.

Timeline - Week 5 - 8 Lent

- **Alternative optimisation algorithms.** Investigate whether a policy-gradient method (e.g. GRPO) can outperform GA in data efficiency or robustness once a stable torque-mapped simulation pipeline is available.
- **Time-varying VMC parameters.** Extend the representation so that stiffness/damping/rest length vary over time or by task phase (e.g. approach vs. grasp vs. lift), rather than remaining constant throughout an episode.
- **Multi-step / hierarchical control.** Introduce a higher-level mechanism to switch between multiple VMC primitives or parameter schedules to execute multi-stage tasks.

5 Risk assessment

R1 (L: High, I: High) Sim to Real Gap

Difficulty in accurately tracking objects pose and shape, robot torque limits, inaccurate contact simulations could all pose a significant challenge. *Mitigation:* There are multiple options for object tracking that can be tested (optitrack, pointcloud). Also we will start with basic shapes with simple geometries. Iteration will allow us to tune simulation parameters to match reality more closely. This will all increase the required time but time pressure is not too high based on the current plan.

R2 (L: Low–Med, I: High) Safety during trials. Optimised parameters may generate unsafe forces which can damage the robot and the object. *Mitigation:* hard caps and termination conditions, conservative bounds, contact-only trials before lift.

R3 (L: Med, I: Med) Compute lim-

its for generalisation Robustness training via GA may be too expensive. *Mitigation:* staged domain randomisation, parallel evaluation, prioritise pose/friction/mass. If none work, can switch to a more generalisable search algorithm.

R4 (L: Med, I: Med) Advanced methods not ready. Alternative optimisers may not converge in time or may not work at all. *Mitigation:* keep as stretch goal; benchmark fairly against GA using identical evaluation.

References

- [1] Markku Suomalainen, Yiannis Karayianidis, and Ville Kyrki. A survey of robot manipulation in contact. *arXiv preprint arXiv:2112.01942*, 2021.
- [2] Jerry Pratt, Chee-Meng Chew, Alonzo Torres, Peter Dilworth, and Gill Pratt. Virtual model control: An intuitive approach for bipedal locomotion. *The International Journal of Robotics Research*, 20(2):129–143, 2001. doi: 10.1177/02783640122067309.
- [3] K. F. Man, K. S. Tang, and S. Kwong. Genetic algorithms: Concepts and applications. *IEEE Transactions on Industrial Electronics*, 43(5):519–534, 1996. doi: 10.1109/41.538609.
- [4] Weizhe Yuan, Rui Chen, Yizhou Li, et al. Group relative policy optimization. *arXiv preprint arXiv:2402.03300*, 2024. Demonstrates stable policy optimisation without a value function, suitable for structured and design-oriented optimisation tasks.
- [5] Anis Sahbani, Sami El-Khoury, and Philippe Bidaud. An overview of 3d object grasp synthesis algorithms. *Robotics and Autonomous Systems*, 60(3):326–336, 2012. doi: 10.1016/j.robot.2011.07.016.
- [6] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm:

6 Appendix

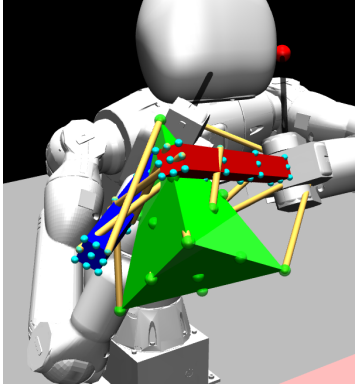


Figure 1: Optimisation pipeline supports mesh objects such as pyramid. sites are automatically added based on faces and edges

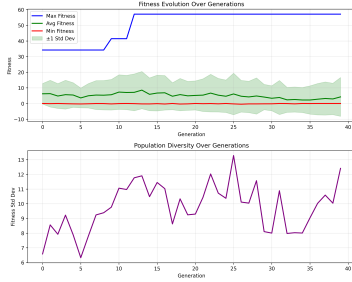


Figure 2: Example evolution statics for a run of the GA algorithm.

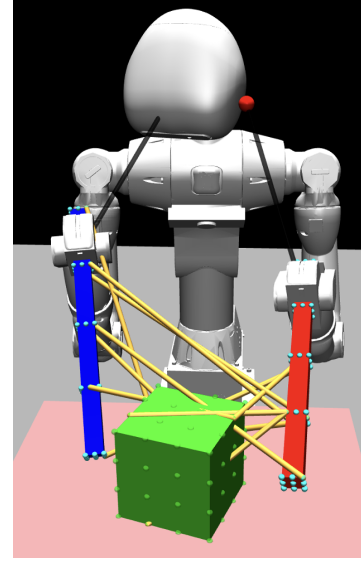


Figure 3: Sciurus17 robot with replaced arms in MuJoCo simulation environment. Green and blue points are sites define on the arms and object. Yellow tendons are connected between sites based on GA encoding

$$\begin{array}{l}
 \text{right to left} \\
 \begin{pmatrix} \text{SL1} & \text{SO1} & s & d & l \\ \text{SL2} & \text{SO2} & s & d & l \\ \text{SL3} & \text{SO3} & s & d & l \end{pmatrix} \\
 \text{left to object} \\
 \begin{pmatrix} \text{SR1} & \text{SO1} & s & d & l \\ \text{SR2} & \text{SO2} & s & d & l \end{pmatrix} \\
 \text{right to object} \\
 \begin{pmatrix} \text{SL1} & \text{SR1} & s & d & l \\ \text{SL2} & \text{SR2} & s & d & l \end{pmatrix}
 \end{array}$$

Figure 4: Genome encoding for the virtual model controller. Each row represents a virtual spring-damper element defined by its endpoint sites and continuous parameters (stiffness s , damping d , rest length l).